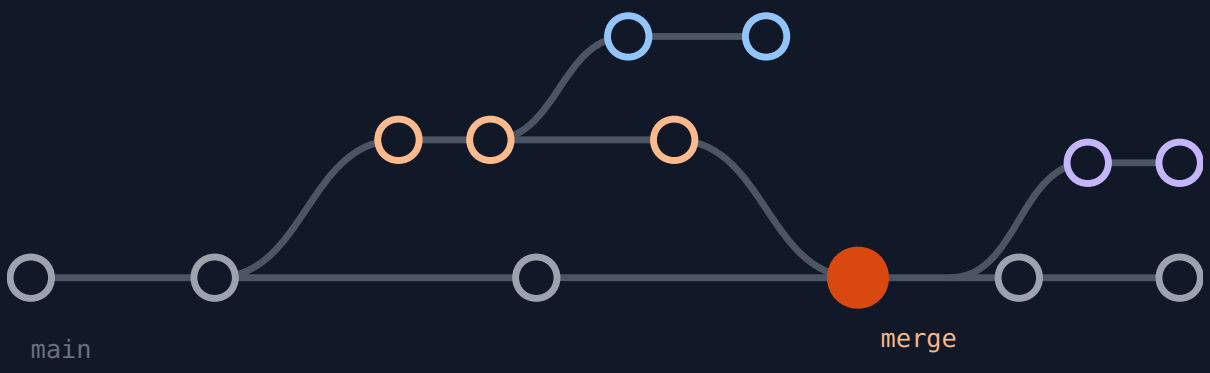
A SHORT, PRACTICAL FIRST GUIDE

Git & GitHub for Beginners

Save your work, undo mistakes, use branches and work in a team, one simple step at a time.



10 short chapters · cheat sheet · 5 practice labs

Commits · History · Undo · Branches · GitHub · Pull requests · Common fixes

October 2026 edition

BEFORE YOU BEGIN

Quick Start: The Core Loop

This page shows the whole journey in one picture. Do not worry if the commands look strange now. Each one is explained, step by step, in the chapters that follow.

```
# One time: who are you?
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main

# Start: get a project
git clone https://github.com/user/project.git  # or: git init
cd project

# Daily loop
git switch -c feature/my-change                # 1. new branch for your work
#    ...edit files...
git status                                     # 2. what changed?
git diff                                       # 3. review the changes
git add .                                      # 4. stage your changes
git commit -m "Add search box to header"       # 5. save a snapshot
git push -u origin feature/my-change           # 6. share it
# 7. open a pull request on github.com

# After the PR is merged
git switch main
git pull                                       # get everyone's latest work
```

Golden rule 1. Run `git status` before and after everything. It tells you where you are and what to do next.

Golden rule 2. Commit small and often. Committed work can almost always be recovered.

Golden rule 3. Never rewrite or force push history that others already have.

What you will be able to do

After chapter	You can...
1 to 4	Install Git, create a project and save snapshots of your work
5 and 6	Read your history and fix the most common mistakes
7	Try ideas safely on branches and solve merge conflicts
8 and 9	Put projects on GitHub and work in a team with pull requests
10	Fix the error messages beginners meet most often

Contents

1 What Are Git and GitHub? 5
The problem 5
Git and GitHub are not the same 5
How to use this guide 5

2 Install and Set Up Git 6
Install Git 6
Tell Git who you are 6
Choose an easy editor 6

3 The Big Idea: Three Areas 8
What is a commit? 8
What is a branch? 9
The four states of a file 9

4 Your First Repository 10
Step 1: create a repository 10
Step 2: check the state with git status 10
Step 3: stage changes with git add 11
Step 4: save a snapshot with git commit 11
Writing good commit messages 11
Ignoring files with .gitignore 11

5 Looking at Changes and History 13
What did I change? git diff 13
What happened before? git log 13
Look at one commit: git show 13
Ways to name a commit 14

6 Fixing Mistakes 15
First, ask one question 15
The mistake fixer table 15
git revert: the safe undo 15
git reflog: your safety net 15
Putting work aside: git stash 16

7 Branches 17
Why use branches? 17
Create and switch branches 17
Merge a branch 17
Merge conflicts 17

8 Connecting to GitHub 19
Create your account 19
Log in from your computer 19
Put your project on GitHub 19
Copy an existing project: git clone 20
Daily sync: push and pull 20
When a push is rejected 20
The README file 21

9 Working Together with Pull Requests 22
What is a pull request? 22
The team workflow, step by step 22
Writing a good pull request 22

Reviewing kindly 23
Issues: a to-do list for your project 23
Contributing to someone else's project: forks 23

10 Common Problems and Fixes 24
"I committed to main, but wanted a new branch" (not pushed) 24
"Push rejected: fetch first" 24
"fatal: refusing to merge unrelated histories" 24
"Authentication failed" or "Support for password authentication was removed" 24
"Permission denied (publickey)" 24
"Your local changes would be overwritten" 24
"I committed a password or API key" 25
"File is larger than 100 MB" 25
"I am totally confused" 25

Appendix A: Cheat Sheet 26

Appendix B: Practice Labs 27

Appendix C: Glossary 28

CHAPTER 1

What Are Git and GitHub?

1.1 The problem

Most people start saving their work like this:

```
project.py
project_v2.py
project_v2_final.py
project_v2_final_REALLY_final.py
```

Soon you cannot tell what changed between copies, who changed it, or which version worked. When two people edit the same file, one person's work gets lost.

Git solves this. It is a **version control system**: it records every change to your files, so you can:

- **see history:** who changed what, when, and why,
- **go back** to any earlier version,
- **try ideas safely** on a separate line of work (a *branch*),
- **work in a team** and combine everyone's changes.

1.2 Git and GitHub are not the same

	Git	GitHub
What it is	A **program** on your computer	A **website** that stores Git projects online
Main job	Tracks changes to your files	Shares projects, reviews code, team work
Needs the internet?	No	Yes
Cost	Free	Free plan for most users

Think of Git as a camera that takes snapshots of your project, and GitHub as an online album where you share them. Other websites like GitLab and Bitbucket do the same job as GitHub.

1.3 How to use this guide

Read the chapters in order, and **type every command yourself** in a practice folder. Git is learned with your hands, not only your eyes.

- Commands are shown in light boxes. Text after `#` is a comment; do not type it.
- Dark boxes labelled **OUTPUT** show what Git prints back.
- Words in angle brackets are placeholders. For `git switch <branch>`, type something like `git switch login-page`.

CHAPTER 2

Install and Set Up Git

2.1 Install Git

Windows. Download *Git for Windows* from `git-scm.com` and run the installer. It includes **Git Bash**, a terminal where you can type the commands in this guide. The default options are fine. On the screen about the initial branch name, choose *Override* and type `main`.

macOS. Open Terminal and type `git --version`. If Git is missing, macOS offers to install it.

Linux (Ubuntu or Debian).

```
sudo apt install git
```

Check that it works:

```
git --version
```

```
OUTPUT
git version 2.43.0
```

Any version from 2.23 or newer works with this guide.

2.2 Tell Git who you are

Git puts your name and email on every snapshot you save. Set them **once**:

```
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

The third line names your first branch `main`. The last line tells `git pull` to combine your work and other people's work with a normal merge. Without it, newer versions of Git stop and ask you to choose.

> **Which email?** Use the same email as your GitHub account, so GitHub can link your work to your profile.

2.3 Choose an easy editor

Sometimes Git opens a text editor so you can write a message. The default is often **Vim**, which confuses beginners. Choose a simpler one:

```
git config --global core.editor "code --wait"   # VS Code
# or
git config --global core.editor "nano"          # simple terminal editor
```

> **Stuck in Vim?** Press `Esc`, type `:wq` and press Enter to save and exit. Type `:q!` to exit without saving.

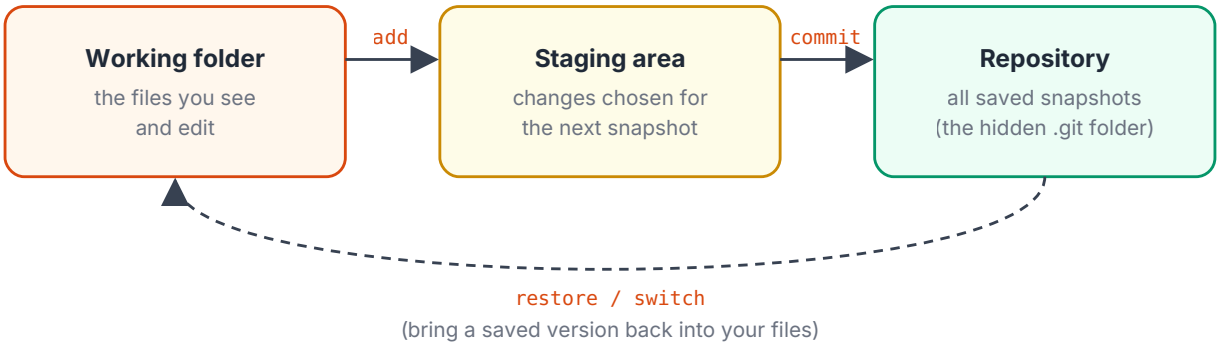
To see all your settings:

```
git config --list
```

CHAPTER 3

The Big Idea: Three Areas

Understand this one picture and Git becomes much easier.



Your work moves from the working folder, to the staging area, to the repository.

1. **Working folder.** Your normal files. You edit them with any editor. Git watches them but saves nothing automatically.
2. **Staging area.** A waiting room. With `git add` you choose which changes go into the next snapshot.
3. **Repository.** With `git commit`, Git saves the staged changes as a permanent snapshot called a **commit**.

Why have a waiting room? So each commit tells one clear story. If you fixed a bug and also changed some colours, you can commit the bug fix first, with its own message, and the colours later.

3.1 What is a commit?

A **commit** is a saved snapshot of your whole project, plus:

- your name and the time,
- a message that explains the change,
- a link to the commit before it.

Each commit gets a unique ID (a *hash*) such as `9fcfb66`. Together the commits form a chain, which is your project's history:



Each commit points back to the one before it. "main" marks the newest.

History is a chain of commits. The branch name "main" points to the latest one.

3.2 What is a branch?

A **branch** is a separate line of work with its own name. The first branch is called `main`. You will learn to create more branches in Chapter 7.

3.3 The four states of a file

`git status` tells you which state each file is in:

State	Meaning
Untracked	A new file Git has never saved
Modified	Changed since the last commit, but not staged
Staged	Marked to go into the next commit
Committed	Safely saved in the repository

Remember

- Edit files, then `git add` to stage, then `git commit` to save a snapshot.
- A commit is a snapshot plus your name, the time and a message.
- Almost everything happens on your own computer. You only need the internet to share with GitHub.

CHAPTER 4

Your First Repository

A **repository** (or *repo*) is a project folder that Git tracks.

4.1 Step 1: create a repository

```
mkdir my-project
cd my-project
git init
```

```
OUTPUT
Initialized empty Git repository in /home/ayesha/my-project/.git/
```

Git creates a hidden `.git` folder where it keeps all the history. **Never delete it**, or you lose every saved version.

4.2 Step 2: check the state with git status

Create two files, then ask Git what it sees:

```
echo "# My Project" > README.md
echo "print('hello')" > app.py
git status
```

```
OUTPUT
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    app.py

nothing added to commit but untracked files present (use "git add" to track)
```

Git tells you where you are and even suggests the next command. **Run `git status` often.** It is the most useful command for beginners.

4.3 Step 3: stage changes with git add

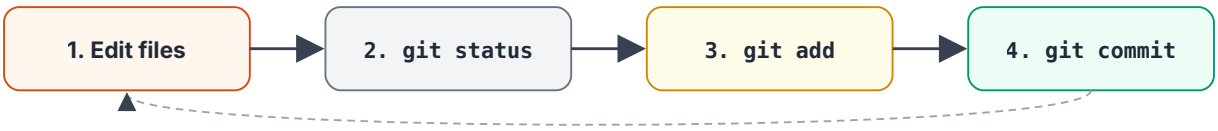
```
git add README.md      # stage one file
git add .              # stage everything in this folder
```

4.4 Step 4: save a snapshot with git commit

```
git commit -m "Add README and hello script"
```

```
OUTPUT
[main (root-commit) 9fcfb66] Add README and hello script
 2 files changed, 2 insertions(+)
 create mode 100644 README.md
 create mode 100644 app.py
```

Done. Your first snapshot is saved. From now on, your daily routine is a simple loop:



The basic loop you will repeat many times a day.

> **Shortcut with a trap.** `git commit -a -m "message"` stages and commits all *changed* files in one step, but it does **not** include *new* files. Use `git add` for new files.

4.5 Writing good commit messages

Your history is a diary of the project. Write messages that your future self will understand.

- Start with a verb, like a command: **Add**, **Fix**, **Remove**, **Update**.
- Keep it short (about 50 characters) and specific.
- One topic per commit.

Weak message	Good message
`update`	`Fix crash when the input file is empty`
`changes`	`Add email check to the sign-up form`
`final!!`	`Update README with install steps`

4.6 Ignoring files with .gitignore

Some files should never be saved in Git: passwords, temporary files, large data, and folders that tools can rebuild. List them in a file named `.gitignore` in the top folder of your project:

.GITIGNORE

```
# Secrets
.env

# Python
__pycache__/
.venv/

# Node.js
node_modules/

# System files
.DS_Store
Thumbs.db

# Large data
data/
*.zip
```

Save and commit the `.gitignore` file itself, so everyone on the team uses it.

Never commit passwords or API keys. If a secret reaches GitHub, bots can find it within minutes. Change the password or key immediately, even if you delete the file later.

Already committed a file by mistake? Adding it to `.gitignore` is not enough. Stop tracking it, but keep it on your disk:

```
git rm --cached .env
git commit -m "Stop tracking .env"
```

Try it. Create a folder `practice` and run `git init`. Make three files and save each one in its own commit with a good message. Then create `notes.tmp`, add `*.tmp` to `.gitignore`, and check with `git status` that `notes.tmp` is no longer listed.

CHAPTER 5

Looking at Changes and History

5.1 What did I change? git diff

```
git diff             # changes you have NOT staged yet
git diff --staged    # changes you HAVE staged (what will be committed)
```

```
OUTPUT
diff --git a/app.py b/app.py
--- a/app.py
+++ b/app.py
@@ -1 +1,2 @@
 print('hello')
+print('bye')
```

Lines with `+` were added. Lines with `-` were removed. Other lines are unchanged context.

> **Good habit:** run `git diff --staged` just before every commit, to check exactly what you are about to save.

5.2 What happened before? git log

```
git log                          # full history (press q to exit)
git log --oneline                # one line per commit
git log --oneline --graph --all  # draw all branches as a picture
git log -5                       # only the last 5 commits
git log -- app.py                # only commits that changed app.py
```

```
OUTPUT
2d08a7e (HEAD -> main) Add login form validation
e94b31c Fix typo in README
7c2e1d0 Add login form
a1f3b2c Initial commit
```

`HEAD` means "where you are now".

5.3 Look at one commit: git show

```
git show e94b31c                 # message and changes of one commit
git show e94b31c:app.py          # app.py as it was in that commit
```

The second form lets you read an old version of a file without changing anything.

5.4 Ways to name a commit

You type	It means
`e94b31c`	A commit by its ID (the first 7 characters are enough)
`HEAD`	The current commit
`HEAD~1`	One commit before the current one
`HEAD~3`	Three commits before the current one
`main`	The newest commit on the branch `main`

CHAPTER 6

Fixing Mistakes

Everyone makes mistakes. The good news: if you **committed** your work at least once, Git can almost always get it back.

6.1 First, ask one question

Have I already pushed this commit to GitHub?

- **No, it is only on my computer:** you can change or remove it freely.
- **Yes, others may have it:** do not remove it. Use `git revert` to add a new commit that undoes it.

6.2 The mistake fixer table

Problem	Command
I want to throw away my edits to a file	`git restore <file>`
I staged a file by mistake	`git restore --staged <file>`
My last commit message has a typo (not pushed)	`git commit --amend -m "New message"`
I forgot a file in my last commit (not pushed)	`git add <file>` then `git commit --amend --no-edit`
Undo my last commit but keep the work (not pushed)	`git reset --soft HEAD~1`
Undo a commit that is already on GitHub	`git revert <commit-id>`
I need an old version of one file	`git restore --source=<commit-id> <file>`
I lost a commit after a reset	`git reflog` (see below)

> `git restore <file>` deletes your **uncommitted** edits for ever. Git never saved them, so it cannot bring them back. If you are not sure, save them on a shelf with `git stash` instead (below).

6.3 git revert: the safe undo

`git revert` does not delete history. It adds a **new** commit that does the opposite of an old one. Because history stays the same, it is safe even after you have pushed.

```
git log --oneline          # find the ID of the bad commit
git revert e94b31c         # create a commit that undoes it
```

6.4 git reflog: your safety net

Git keeps a private diary of every place you have been. If a commit seems lost, look there:

```
git reflog
```

```
OUTPUT
2d08a7e HEAD@{0}: reset: moving to HEAD~1
9c41f0b HEAD@{1}: commit: Add payment page
2d08a7e HEAD@{2}: commit: Add product list
```

The "lost" commit `9c41f0b` is still there. Save it on a new branch:

```
git branch rescue 9c41f0b
```

6.5 Putting work aside: git stash

You are in the middle of something, but you need a clean folder right now (for example, to switch branches). Put your unfinished changes on a shelf:

```
git stash          # shelve your uncommitted changes
# ...do something else...
git stash pop      # bring them back
git stash list     # see everything on the shelf
```

Remember

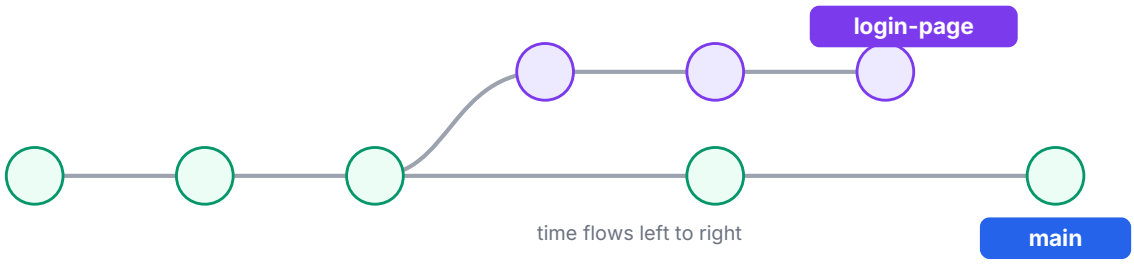
- Commit often. Committed work is very hard to lose.
- Not pushed yet? You can amend or reset. Already pushed? Use `git revert`.
- `git reflog` finds commits that seem lost.

CHAPTER 7

Branches

7.1 Why use branches?

A **branch** lets you work on a new feature or try an idea **without touching** the working version on `main`. If the idea works, you merge it in. If not, you delete the branch and nothing is harmed.



The branch login-page splits from main. Both can get new commits without affecting each other.

7.2 Create and switch branches

```
git branch                  # list branches (* shows the current one)
git switch -c login-page    # create a new branch and move to it
git switch main             # move back to main
git switch login-page       # move to an existing branch
```

When you switch, Git changes the files in your folder to match that branch. Commit or stash your changes before switching.

> Older tutorials use `git checkout -b login-page` and `git checkout main`. They still work. `git switch` is the newer, clearer command.

7.3 Merge a branch

When the work on the branch is finished, go to the branch you want to update, and merge:

```
git switch main
git merge login-page
git branch -d login-page    # delete the branch you no longer need
```

You always merge **into** the branch you are on.

7.4 Merge conflicts

A **conflict** happens when two branches changed **the same lines** of the same file in different ways. Git cannot choose, so it asks you. This is normal. It does not mean you broke anything.

```
OUTPUT
CONFLICT (content): Merge conflict in config.py
Automatic merge failed; fix conflicts and then commit the result.
```

Open the file. Git has marked both versions:

```
<<<<<<< HEAD
timeout = 30
=======
timeout = 60
>>>>>>> login-page
```

- The top part (after `<<<<<<< HEAD`) is the version on your current branch.
- The bottom part (before `>>>>>>> login-page`) is the version from the other branch.

To fix it:

1. Edit the file so it has the correct final code, for example `timeout = 60`.
2. Delete the three marker lines (`<<<<<<<`, `=======`, `>>>>>>>`).
3. Stage and commit:

```
git add config.py
git commit
```

Want to give up and go back? Run `git merge --abort`.

> **VS Code makes conflicts easy.** It highlights each conflict and shows buttons: *Accept Current*, *Accept Incoming* and *Accept Both*.

> **Try it: create a conflict on purpose.** On `main`, create `color.txt` containing `blue` and commit. Create a branch `paint`, change the line to `green`, commit. Switch back to `main`, change it to `red`, commit. Now run `git merge paint`, fix the conflict to `purple`, and commit. See the result with `git log --oneline --graph`.

CHAPTER 8

Connecting to GitHub

8.1 Create your account

1. Sign up at `github.com`. Choose a professional username; it appears in all your project links.
2. Add a photo, your name and a short bio.
3. Turn on **two-factor authentication (2FA)** in Settings, then *Password and authentication*. Save your recovery codes.

> **Students** can apply for free extra benefits at `education.github.com`.

8.2 Log in from your computer

GitHub does **not** accept your normal password for Git commands. The easiest way to log in is the **GitHub CLI** (`gh`), a free tool from `cli.github.com`:

```
gh auth login
```

Answer the questions: choose *GitHub.com*, then *HTTPS*, then *Login with a web browser*. Copy the one-time code, paste it in the browser page, and you are done. Git can now talk to GitHub.

Alternative: an SSH key

Many developers use an SSH key instead. Create it once per computer:

```
ssh-keygen -t ed25519 -C "you@example.com"   # press Enter to accept defaults
cat ~/.ssh/id_ed25519.pub                     # show the PUBLIC key
```

Copy the printed line. On GitHub go to Settings, then *SSH and GPG keys*, then *New SSH key*, and paste it. Test it:

```
ssh -T git@github.com
```

```
OUTPUT
Hi ayesha! You've successfully authenticated, but GitHub does not provide shell access.
```

> Share only the file ending in `.pub`. The other file (`id_ed25519`) is your **private** key. Never send it to anyone.

8.3 Put your project on GitHub

Step 1. On GitHub, click **+** (top right), then **New repository**. Give it a name, choose *Public* or *Private*, and **do not** tick "Add a README" (your project already has files). Click **Create repository**.

Step 2. GitHub shows the commands. In your project folder, run:

```
git remote add origin https://github.com/ayesha/my-project.git
git branch -M main
git push -u origin main
```

- `remote add origin` saves the GitHub address under the short name `origin`.
- `push` uploads your commits.
- `-u` links your `main` to GitHub's `main`. After this, plain `git push` is enough.

Refresh the GitHub page: your files and history are online.

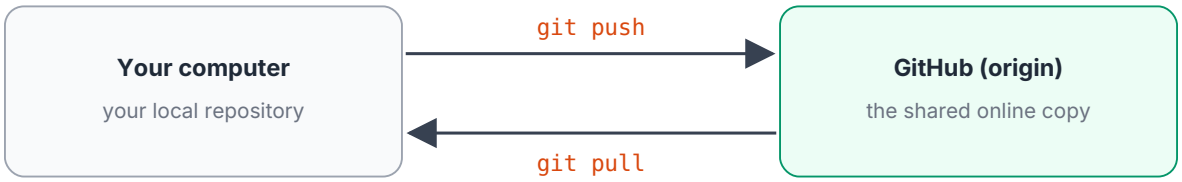
8.4 Copy an existing project: git clone

To download a project from GitHub with its full history:

```
git clone https://github.com/ayesha/my-project.git
cd my-project
```

Cloning sets up `origin` for you automatically.

8.5 Daily sync: push and pull



push sends your commits to GitHub. pull brings other people's commits to you.

Command	What it does
`git push`	Upload your new commits to GitHub
`git pull`	Download new commits from GitHub and add them to your branch
`git fetch`	Download new commits, but do not change your files yet (always safe)

A good routine: **pull before you start working, push when you finish.**

8.6 When a push is rejected

```
OUTPUT
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:ayesha/my-project.git'
```

This means someone else pushed first, and GitHub has commits that you do not have. Git refuses to overwrite their work. Fix:

```
git pull         # get their work (fix any conflict, then commit)
git push         # now your push works
```

> Do not "solve" this with `git push --force`. That deletes your teammates' work from GitHub.

8.7 The README file

`README.md` is the front page of your project on GitHub. A good README says:

1. **What** the project is, in one or two sentences.
2. **How to install and run it**, with exact commands.
3. **How to use it**, with a short example or screenshot.
4. The **license**, if others may reuse the code.

README files are written in **Markdown**, a simple text format:

```
# Project Title
Some **bold** text and some *italic* text.

## Installation
- Step one
- Step two

[A link](https://github.com)
```

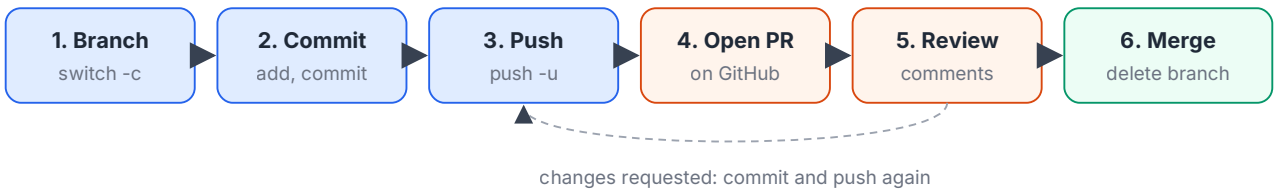
CHAPTER 9

Working Together with Pull Requests

9.1 What is a pull request?

A **pull request (PR)** is a request to merge your branch into `main`, with a web page where teammates can see your changes, comment and approve. It is the standard way teams work on GitHub.

9.2 The team workflow, step by step



The pull request cycle used by most teams.

```
# 1. Start from the latest main
git switch main
git pull

# 2. Make a branch and do your work
git switch -c fix-login-button
git add .
git commit -m "Fix login button colour on mobile"

# 3. Upload the branch
git push -u origin fix-login-button
```

4. Open the PR. On GitHub, a yellow banner appears with a **Compare & pull request** button. Click it, write a clear title and description, and click **Create pull request**.

5. Review. Teammates read the *Files changed* tab and leave comments. To make changes, just commit and push again on the same branch; the PR updates automatically.

6. Merge. When it is approved, click **Merge pull request** on GitHub, then delete the branch. Update your computer:

```
git switch main
git pull
git branch -d fix-login-button
```

9.3 Writing a good pull request

- **Title:** short and specific, like "Fix login button colour on mobile".
- **Description:** what you changed, why, and how to test it.

- **Link the issue:** writing `Closes #12` closes issue number 12 automatically when the PR is merged.
- **Keep it small.** Small PRs get faster and better reviews.

9.4 Reviewing kindly

- Comment on the code, not the person: "This might fail when the list is empty" is better than "You made a mistake".
- Ask questions when you are not sure.
- Say what is good, too.

9.5 Issues: a to-do list for your project

Issues (the *Issues* tab) track bugs, ideas and tasks. Each one gets a number, like `#12`. You can add **labels** (`bug`, `enhancement`), **assign** a person, and discuss in comments. A good bug report says what you expected, what happened, and how to reproduce it.

9.6 Contributing to someone else's project: forks

If you cannot push to a project (for example, an open source project), you contribute through a **fork**, your own copy of it on GitHub:

1. Click **Fork** on the project page.
2. Clone **your fork**, make a branch, commit and push to your fork.
3. Open a pull request from your fork to the original project.

Read the project's `CONTRIBUTING.md` first, and look for issues labelled `good first issue`.

CHAPTER 10

Common Problems and Fixes

Before anything else, run these two commands. They explain most problems:

```
git status
git log --oneline --graph --all -10
```

10.1 "I committed to main, but wanted a new branch" (not pushed)

```
git branch my-feature        # the new branch keeps your commit
git reset --hard HEAD~1      # move main back one commit
git switch my-feature        # continue working here
```

10.2 "Push rejected: fetch first"

Someone pushed before you. Run `git pull`, fix any conflict, then `git push`.

10.3 "fatal: refusing to merge unrelated histories"

You created the GitHub repository *with* a README and also ran `git init` on your computer. Join the two histories once:

```
git pull origin main --allow-unrelated-histories
git push -u origin main
```

10.4 "Authentication failed" or "Support for password authentication was removed"

You typed your GitHub password. GitHub does not accept it for Git. Run `gh auth login`, or set up an SSH key (Chapter 8).

10.5 "Permission denied (publickey)"

Your SSH key is not set up. Check that you added the `.pub` key to GitHub, and test with `ssh -T git@github.com`.

10.6 "Your local changes would be overwritten"

Git will not switch branches or pull because you have unsaved edits. Either commit them, or stash them:

```
git stash
git pull           # or git switch other-branch
git stash pop
```

10.7 "I committed a password or API key"

1. **First, change the password or delete the key** on the service where it was created. Assume it is already stolen.
2. Remove the file from Git and ignore it:

```
git rm --cached .env
echo ".env" >> .gitignore
git add .gitignore
git commit -m "Remove secret file and ignore it"
```

The secret is still in older commits. For a public repository, ask an experienced person for help to clean the history.

10.8 "File is larger than 100 MB"

GitHub blocks files over 100 MB. Do not commit large data, videos or model files. Add them to `.gitignore`. If the big file is in your last commit (not pushed): `git rm --cached big-file`, then `git commit --amend --no-edit`.

10.9 "I am totally confused"

- `git merge --abort` or `git rebase --abort` cancels a merge or rebase in progress.
- Copy your project folder somewhere safe before trying more fixes.
- Search the exact error message online. Someone has seen it before.
- Last resort: clone a fresh copy from GitHub and copy your changed files into it.

APPENDIX A

Cheat Sheet

Set up (once)

Command	Purpose
`git config --global user.name "Name"`	Set your name
`git config --global user.email "you@x.com"`	Set your email
`gh auth login`	Log in to GitHub from your computer

Start

Command	Purpose
`git init`	Make the current folder a repository
`git clone <url>`	Download a project from GitHub

Save work

Command	Purpose
`git status`	What has changed?
`git add <file>` / `git add .`	Stage one file / everything
`git commit -m "message"`	Save a snapshot
`git diff` / `git diff --staged`	See unstaged / staged changes
`git log --oneline`	Short history

Branches

Command	Purpose
`git branch`	List branches
`git switch -c <name>`	Create a branch and move to it
`git switch <name>`	Move to a branch
`git merge <name>`	Merge a branch into the current one
`git branch -d <name>`	Delete a merged branch

GitHub

Command	Purpose
`git remote add origin <url>`	Connect a local project to GitHub
`git push -u origin main`	First upload of a branch
`git push`	Upload new commits
`git pull`	Download and merge new commits

Undo

Command	Purpose
`git restore <file>`	Throw away edits (permanent)
`git restore --staged <file>`	Unstage, keep the edits
`git commit --amend`	Fix the last commit (not pushed)
`git revert <commit-id>`	Undo a pushed commit safely
`git stash` / `git stash pop`	Shelve changes / bring them back
`git reflog`	Find "lost" commits

APPENDIX B

Practice Labs

Do these in order, in a practice folder. Each lab ends with a **check** that proves you succeeded.

Lab 1. First commits

1. Set your name and email.
2. Create a folder `recipes` and run `git init`.
3. Create `README.md` and commit it. Create `rice.md` and commit it separately.
4. Edit `rice.md`, look at the change with `git diff`, then commit.

Check: `git log --oneline` shows three commits with clear messages.

Lab 2. Ignore and undo

1. Create `notes.tmp` and add `*.tmp` to `.gitignore`. Commit the `.gitignore`.
2. Edit `rice.md`, then throw the edit away with `git restore`.
3. Make a commit with a typo in its message, then fix it with `git commit --amend`.

Check: `git status` says "nothing to commit", and `git log` shows the corrected message.

Lab 3. Branch and conflict

1. Create a branch `spicy`, change one line of `rice.md`, and commit.
2. Switch to `main`, change the same line differently, and commit.
3. Merge `spicy` into `main`, fix the conflict, and commit.

Check: `git log --oneline --graph` shows two lines joining together.

Lab 4. Go online

1. Log in with `gh auth login` (or set up an SSH key).
2. Create an empty repository on GitHub and push `recipes` with `git push -u origin main`.
3. Edit `README.md` on the GitHub website and commit there.
4. On your computer, run `git pull`.

Check: your local `README.md` now has the change you made on the website.

Lab 5. Pull request with a friend

1. Add your friend as a collaborator (Settings, then *Collaborators*).
2. Each of you makes a branch, adds a new recipe file and opens a pull request.
3. Review each other's PR and leave one helpful comment.
4. Merge both PRs and delete the branches.

Check: both new recipe files are on `main` on GitHub, and `git pull` brings them to your computer.

APPENDIX C

Glossary

Word	Meaning
Branch	A separate line of work with its own name, like `main` or `login-page`.
Clone	A full copy of a GitHub project on your computer.
Commit	A saved snapshot of your project with a message.
Conflict	Two changes to the same lines that Git cannot combine by itself.
Diff	The list of lines that changed between two versions.
Fork	Your own copy of someone else's project on GitHub.
HEAD	"Where you are now": your current branch and commit.
Issue	A GitHub item for a bug, task or idea.
Merge	Combine the work of one branch into another.
Origin	The usual name for your project's GitHub address.
Pull	Download new commits from GitHub and add them to your branch.
Pull request (PR)	A request, on GitHub, to merge your branch after review.
Push	Upload your new commits to GitHub.
Remote	A copy of the repository on another computer, usually GitHub.
Repository (repo)	A project folder tracked by Git, with all its history.
Revert	A new commit that undoes an older one.
Staging area	The waiting room where you choose what goes into the next commit.
Stash	A shelf for unfinished changes.
Working folder	The normal files on your disk that you edit.

What next?

You now know enough to use Git and GitHub every day. When you are comfortable, move on to rebasing, GitHub Actions for automatic testing, releases and GitHub Pages. They are all covered in *The Complete Git & GitHub Guide*. Free practice: the visual game `learngitbranching.js.org` and the courses at `skills.github.com`.

Keep three habits: run `git status` often, commit small and often, and pull before you start working.